BUILDING GENAIPROS · TOPIC 01

API Styles in the Agent Era

MCP is not a new style — it is a contract layered on 30 years of plumbing

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

What Is an API Style?

A shape of communication, not a product or a trend

An API style is a paradigm — a set of patterns, protocols, and practices for exposing application interfaces. It is not a vendor. It is not a technology. It is the shape your traffic has. Four shapes cover virtually all distributed systems: one-shot synchronous, streamed, bidirectional, and async-decoupled. Choose the shape your traffic has and the style usually picks itself. Teams that choose by familiarity or trend, then force their traffic to fit, produce working code and broken architectures.

The style question is really a traffic-shape question — answer that first.

The Seven Styles at a Glance

Every AI architecture is a composition of several of these

Style	Protocol	Shape	Typical AI Use
REST	HTTP/1.1	Sync one-shot	LLM chat APIs — OpenAI, Anthropic
GraphQL	HTTP / WS	Sync one-shot	RAG context shaping, no overfetch
gRPC	HTTP/2	Sync + streaming	Internal serving — Triton, vLLM
SSE	HTTP	Server-push stream	Token-by-token chat streaming
WebSocket	TCP/WS	Bidirectional duplex	Voice agents, realtime multimodal

No single AI product uses just one style — map all seven before you architect.

Request-Response Styles

The synchronous trio powering most AI frontends today

RE

REST

Resource URLs plus fixed verbs — GET, POST, PUT, DELETE. JSON on the wire. Every LLM chat API — OpenAI, Anthropic, HF Inference — is a REST POST. Stateless, CDN-friendly, debuggable with curl.

Default style for every LLM chat call.

GQ

GraphQL

Client specifies exactly which fields it wants from a typed schema. One endpoint, one round trip, zero overfetching. RAG pipelines use it to pull only the fields the LLM needs, conserving context window and token cost.

Precision context retrieval for RAG pipelines.

gR

gRPC

Proto-defined functions, binary Protocol Buffers, HTTP/2 multiplexing. Four modes including bidirectional streaming. Triton, TF Serving, and vLLM all speak gRPC internally for high-throughput inference traffic.

The spine for internal model serving at scale.

REST is for the edge; gRPC is for the spine. Do not swap them.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

Streaming and Async Styles

Four styles that handle time — tokens, audio, events, and queues

SE

Server-Sent Events

Long-lived HTTP response; server pushes data events one by one. Auto-reconnect, proxy-friendly. Every token-streaming chat UI — OpenAI, Anthropic, HF — runs on SSE under the hood. Server-to-client only.

The wire under every streaming chat UI.

WS

WebSocket

Full-duplex persistent connection started via HTTP Upgrade. Both sides send frames freely. Low per-message overhead. OpenAI Realtime API, voice agents, and live multimodal sessions all require WebSocket.

Required for any realtime bidirectional session.

WH

Webhooks

HTTP POST from server to a registered URL when an event fires. OpenAI batch API, fine-tune, and transcription jobs all signal completion this way.

Right pattern for long-running job callbacks.

BR

Broker / Queue

Kafka, GCP Pub/Sub, RabbitMQ — producers and consumers decouple entirely. Fan-out is free; retries and dead-letter queues are operational primitives. Multi-agent task distribution lives here.

Decouple scale from timing; standard for agent fanout.

Streaming and async styles are where AI systems scale — and where most bugs hide.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

Anatomy of a Streaming Chat Call

REST out, SSE back — two styles composing into one user experience

```
POST /v1/messages              (REST)
Content-Type: application/json

{ "model": "claude-opus-4-7",
  "stream": true,
  "messages": [
    {"role": "user",
     "content": "Explain MCP"} ]
}

--- response ---               (SSE)
Content-Type: text/event-stream

data: {"delta":{"text":"MCP "}}
data: {"delta":{"text":"is a "}}
data: [DONE]
```

REST for intent, SSE for delivery

The request is REST — a synchronous HTTP POST with a JSON body and a bearer token. The response is SSE — a long-lived stream of data events, one per token group, closing with a DONE marker. Same TCP connection; two distinct styles; two different trade-off profiles. Recognising this composition is the key to building chat UIs that feel fast rather than laggy. Nothing about this is new — the styles have been in production for years.

One connection, two styles — the prototypical AI composition pattern.

The M x N Integration Problem

Why 70% of AI codebases became glue code before MCP

The combinatorial trap

- 3 AI tools x 3 data sources = 9 adapters
- Add one tool — write 3 more adapters
- Add one source — write 4 more adapters
- Context lost at every schema translation hop
- 70% of AI codebase is glue not product
- Each adapter is its own auth and audit surface

Symptoms in production

- Chatbot knows purchase history; recommender does not
- Three tools, three summaries of the same record
- Fine-tune dataset schemas mismatched across sources
- Auth policies duplicated per integration pair
- No unified audit log across the glue layer
- One broken adapter silently corrupts downstream agents

M x N is not a scaling problem — it is architectural debt that compounds with every new tool.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

What MCP Actually Is

A standardised contract layered on existing plumbing — not a new API style

The Model Context Protocol is an open standard introduced by Anthropic in November 2024, contributed to the Linux Foundation in December 2025, and adopted by Anthropic, OpenAI, and Google DeepMind. It defines how AI applications expose tools, resources, and prompts to LLMs — and how LLMs invoke them at runtime. The architectural shift is M+N: each tool implements MCP once, each data source exposes one MCP server, and any compliant client can use any server. Under the hood it speaks JSON-RPC 2.0 over stdio or streamable HTTP+SSE — familiar two-decade-old plumbing, with a new semantic layer on top.

MCP is the USB-C moment for AI tooling — one connector, any device.

MCP Under the Hood — Four Layers

Innovation lives only in the semantic layer; everything else is reused infrastructure

L4 Application — The Host

Your LLM-powered app — Claude Desktop, ChatGPT, Cursor, or a custom agent. It manages the LLM session, picks which MCP servers to connect, and owns the trust boundary.

Trust decisions live at the host, not the server.

L3 MCP Primitives — The New Layer

The only genuinely new layer in the stack. Server-side: tools, resources, prompts. Client-side: sampling, roots, elicitation. Everything below this layer is reused infrastructure.

Tools are verbs; resources are nouns; prompts are workflows.

L2 JSON-RPC 2.0 — The Envelope

Standard message format — methods, params, results, errors. In production since 2005. Stateless by default; sessions opt in via the Mcp-Session-Id header.

Twenty-year-old plumbing. Zero reinvention required.

L1 Transport — Two Modes

Stdio for local subprocess servers — the host launches the server and talks via stdin and stdout. Streamable HTTP+SSE for remote servers — an HTTPS endpoint secured with OAuth 2.1 and PKCE.

Local = stdio; remote = HTTP+SSE + OAuth 2.1.

Strip the primitives layer and what remains is infrastructure every backend engineer already runs.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

MCP Primitives — What Servers Expose

Three server-side primitives the LLM discovers and invokes at runtime

TL

Tools — Verbs

Callable functions with a name, description, and JSON Schema input. The LLM reads descriptions at runtime, picks a tool, and invokes it. Accuracy drops sharply above roughly ten tools per server — narrow names beat broad ones.

Precision naming is the highest-leverage MCP skill.

RS

Resources — Nouns

Readable context identified by URI — a file, a database row, an API response. Read-only from the model's perspective. Subscribable, so the host is notified when content changes.

Resources are what the LLM reads; tools are what it does.

PR

Prompts — Workflows

Reusable conversation templates the server publishes. Invoked by the user via slash command, not by the LLM. They return a fully-formed conversation seed — server-defined best practices.

Prompts encode team knowledge as repeatable workflows.

In-band runtime discovery is what separates MCP from every hand-written OpenAPI spec before it.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

Three Contracts — One Wire Family

Traditional API vs MCP vs A2A — same plumbing, different audience

Axis	Traditional API	MCP	A2A
Client type	Human developer	LLM at runtime	Autonomous agent
Discovery	OpenAPI docs — out of band	list_tools — in band	agent-card.json — in band
State model	Stateless, sessions opt-in	Stateless calls, sessions opt-in	Stateful tasks, long-running
Integration cost	M x N custom adapters	M+N — one server, many clients	M+N — at the agent layer

Dynamic in-band discovery is the architectural shift that unites MCP and A2A.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

Meet A2A — MCP's Sibling Protocol

Google + 50 launch partners, April 2025 — in production at 150+ orgs by Cloud Next 2026

AC

Agent Card

JSON metadata served at /.well-known/agent-card.json. Describes capabilities, skills, transport, and security schemes. The A2A analog of MCP list_tools — in-band runtime discovery. Cards can be cryptographically signed.

Discovery without a hand-wired registry.

TK

Tasks and Artifacts

A stateful work unit with a unique ID and lifecycle — submitted, in-progress, completed, failed. Tasks run minutes to days. Artifacts are tangible deliverables. MCP tool calls are one-shot; A2A tasks are workflows.

Tasks are the primitive MCP deliberately omits.

WR

Three Wire Shapes

A2A composes the seven styles beneath it. Request and response polling is REST-shaped. Progress streaming uses SSE. Completion callbacks are webhook-shaped. One protocol, three patterns, zero new transport.

New protocols layer on top — they do not replace.

A2A lets Salesforce Agentforce hand off to Vertex AI to ServiceNow — different vendors, real production traffic.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

Production Challenges — Security

The failure modes burning teams in 2025 MCP deployments

Prompt injection via tool output

- Tool returns attacker-controlled text
- LLM reads it as a trusted instruction
- Largest attack class in MCP deployments today
- Action-Selector — only predefined actions allowed
- Plan-Then-Execute — plan locked before tools run
- Dual-LLM — privileged planner, sandboxed processor

Excessive tool authorization

- MCP inherits whatever scopes the user granted
- No least-privilege enforcement by default
- Narrow send_email_to_team beats broad send_email
- Map-Reduce — untrusted inputs in isolated subagents
- Code-Then-Execute — LLM writes code, code runs deterministically
- Context-Minimization — strip original prompt after step one

Security in MCP is a design problem — narrow tools and elicitation are the primary mitigations.

Production Challenges — Reliability and Ops

Four failure modes that compound across multi-agent chains

H1 Hallucinated Tool Results

Agent reports success when the tool actually timed out. Downstream agents act on the lie; errors cascade. Fix: cite raw tool output, log executions separately, require structured success and failure indicators.

Never let an agent paraphrase a tool result.

H2 Wrong Tool Selection

With 20 similar tools the LLM picks the wrong one. Accuracy in current frontier models drops sharply above roughly ten tools per server. Fix: small surfaces, distinct names, intent guidance in the prompt.

Keep MCP server surfaces under 10 tools.

H3 Schema Drift

An MCP server adds a required field; existing clients break at runtime. Fix: version the server, prefer additive-only changes, test compatibility in CI.

Treat MCP schemas like any versioned API contract.

H4 Chatty Tool Cost Blowup

Each tool call grows the conversation context. A 20-step agent pays token cost scaling with the square of conversation length. Fix: paginate output, return URIs not full payloads, cache the stable prefix.

Token cost grows quadratically with chatty calls.

Current research cites 40-80% task failure rates for multi-agent systems — instrument everything from day one.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

The Hyperscaler Ecosystem in 2026

Protocol is commoditized — the moat is runtime, bridges, identity, and registries

RT

Runtime Hosts

AWS Bedrock AgentCore Runtime — ARM64 microVMs, sticky sessions. Azure AI Foundry Agent Service — Entra identity, OBO OAuth. Google Vertex AI Agent Engine — deploy ADK agents from a Python script. Cloudflare Workers — cheapest global distribution.

Pick by where your data and identity already live.

BR

API-to-MCP Bridges

Google Apigee auto-exposes any managed API as an MCP server, inheriting existing rate limits. AWS Bedrock AgentCore Gateway wraps REST APIs as MCP tools. Azure API Center catalogs MCP servers with Entra governance.

Enterprise API gateway is the new MCP server factory.

ID

Identity and Registries

OAuth 2.1 + PKCE + Dynamic Client Registration is the 2026 baseline. Providers: Microsoft Entra, Auth0, Stytch. Registries: the Anthropic directory, Microsoft Foundry Add Tools, Google Vertex Agent Garden, Zapier AI Actions.

OAuth 2.1 + DCR is the baseline for remote servers.

Linux Foundation governs both MCP and A2A — single-vendor capture is structurally off the table.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

Decision Framework — Which Contract?

Match traffic shape to protocol before writing a single line of integration code

Traffic Shape	Use	Why	Common Mistake
One-shot ask, one answer	REST	Stateless, cacheable, debuggable	Adding streaming when polling works
Server streams over time	SSE	Proxy-friendly, auto-reconnect	WebSocket for one-way push
Both sides talk continuously	WebSocket	Full-duplex, low overhead	REST polling for realtime audio
Decouple sender from receiver	Broker	Free fan-out, retries, DLQs	Chaining webhooks instead of a queue
LLM calls your service	MCP	In-band discovery, M+N cost	Custom REST adapter per tool pair

MCP and A2A layer on top of the seven styles — they do not replace the plumbing beneath them.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

KEY TAKEAWAYS

Five Things Worth Keeping

01 API styles are communication shapes. Pick the shape your traffic has and the style follows. Forcing fit produces working code and broken architectures.

02 Real AI systems compose styles. REST out plus SSE back for chat. WebSocket for voice. gRPC for internal serving. Broker for agent fanout. Nothing in production is single-style.

03 MCP is not a new API style. It is JSON-RPC 2.0 over stdio or HTTP+SSE with a semantic layer on top — tools, resources, prompts, in-band discovery. Same plumbing, new vocabulary.

04 A2A is MCP's sibling for agent-to-agent work. It adds stateful tasks, lifecycle states, and artifacts — the primitives MCP leaves out. Real systems need both, and the two compose.

05 Dynamic in-band discovery is the architectural shift. OpenAPI is read by humans before code ships. MCP and A2A contracts are introspected by models at runtime. That eliminates the glue layer.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros

The Wire Is Settled

Now build on it

GenAIPros wraps its GCP-hosted RAG pipeline as an MCP server — in-band discovery, zero hand-wired clients, every content source and retrieval tool behind one standard contract. A2A enters the picture the moment the ingestion agent and the summarisation agent must hand work across a task lifecycle longer than a tool-call timeout — that boundary is exactly the line this deck traces. MCP for LLM-to-tool. A2A for agent-to-agent. The seven styles underneath both. Follow the build at genaipros.com.

Kamaljeet S Kharbanda
AI Advisor & Instructor @ AIMLEngineers.com · Building GenAIPros